COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Acelajado, John Vince D. Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **[ACTIVITY NAME] Parallelism and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B

Table 13-1. Simple One-Threaded Example

Screenshot

Code:

```
#include <iostream>
#include <thread>

void print(int n, const std::string &str){
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main(){
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
return 0;
}
```

Output:

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

Acelajado: Based on this procedure part about “Simple One-Threaded” I analyze that this program uses a thread library where the print function shows a number and also a word. In the main() part this makes a thread called t1 that runs it with 10 and also T.I.P. we used the t1.join() so that the program will wait for its thread to finish it first then the output will print its integer and output. Based on this the single thread does only one job running so that there is nothing that can get mixed up together and the output always be the same.

	Pangilinan: When I analyzed this code, I saw the program creates only one thread then run it to print the function. When you run it, the thread is given the number 10 and the word “T.I.P.”, and the program waits for the thread to finish before it stops. Since there is only one thread in the code, the output prints in the same order every time you run it.

Table 13-2. Multithreaded Example.

Screenshot	**Code:**

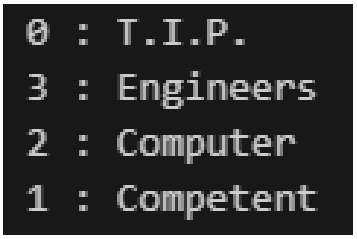
```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
    threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

	Output: 
Analysis	**Acelajado:** Based on this procedure from what I understand and analyze is that we created four words which consist of an individual thread that means that four of them print at the same time. Then every run of the code the execution of it changes every time because of what it does the computer decides which will go first or finish first.. **Pangilinan:** In this part there are four threads that are on loop, one for each word, each thread prints its own number and word and the order is not guaranteed, so once you run it again you can get a different output. Another thing is the threads all start at the same time and it decides which goes first. In our screenshot both runs happened to print 0 to 3 in order, but another run can be different.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

1. Write a definition of multithreading and its advantages/disadvantages.

Pangilinan: Multithreading is an operating system feature that allows a single process to split into multiple concurrent execution threads sharing the same memory and resources. Its advantages include improved execution speed, enhanced application responsiveness, and reduced context-switching time compared to full processes. Its disadvantages include performance degradation if thread counts are unoptimized, complex debugging cycles, and memory management overhead. **Reference:** Unstop. *Multithreading In Operating Systems - Types, Pros, Cons And More*. https://unstop.com/blog/multithreading-in-os	**Acelajado:** Multithreading is a programming technique that allows a single program to execute multiple threads within a single process, with each thread having its own separate stream of program instructions and with all threads sharing the same address space and being able to communicate with each other using shared data. In C++ it is supported by the standard library, in the std::thread (Stroustrup, 2022). The benefits include that it is more responsive, as slower work, like operating on a file or a network, can be performed in the background; more efficient use of multi-core processors to complete work faster, and lower communication costs between different threads than between different processes due to sharing memory. It has disadvantages, such as crash results, data races, and deadlocks due to

	the shared data accessed by multiple threads without synchronization, and programs are more difficult to write, test, and debug because the behaviour is not deterministic (Stroustrup, 2022). Switching between threads adds overhead as does creating threads. **Reference:** Stroustrup, B. (2022). *A Tour of C++* (3rd ed.). Addison-Wesley Professional. https://www.stroustrup.com/tour3.html ExplainThis. (2026, February 21). *What's the difference between concurrency and parallelism?* https://www.explainthis.io/en/swe/concurrency-vs-parallelism

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Pangilinan: Multithreading is applied when workloads can be decoupled into independent execution streams. The first use-case is increasing application responsiveness, such as allowing a user to interact with a web browser while a video or file downloads in a background thread. The second use-case is efficient resource sharing, where threads share parent process memory directly without needing heavy inter-process communication mechanisms. The third use-case is leveraging multi-core microprocessor architectures, where independent threads execute on separate CPU cores simultaneously to maximize scalability and system throughput. **Reference:** GeeksforGeeks. *Benefits of Multithreading in Operating System*. https://www.geeksforgeeks.org/operating-systems/benefits-of-multithreading-in-operating-system/	**Acelajado:** There are three use-cases in which multithreading is appropriate since a program contains work that could be done independently. The first is GUI apps, where one thread is used for GUI operations, and others for loading files or getting network data, and the GUI doesn't lock up. The second case is when there are no servers, such as web or game servers, so that every client request is handled by a new thread or thread pool instead of waiting in line, which works well for jobs that spend a large portion of their time waiting for I/O (ExplainThis, 2026). The third use case is data processing and scientific computing tasks, in which cpu-intensive tasks like image processing or matrix calculations are broken down into threads, each of which is executed on a different core, thus reducing the overall execution time (Stroustrup, 2022). **Reference:** Stroustrup, B. (2022). *A Tour of C++* (3rd ed.). Addison-Wesley Professional. https://www.stroustrup.com/tour3.html ExplainThis. (2026, February 21). *What's the difference between concurrency and parallelism?* https://www.explainthis.io/en/swe/concurrency-vs-parallelism

3. Differentiate between parallelism and concurrency.

Pangilinan: Concurrency describes managing multiple tasks that make progress over overlapping time periods, which can be accomplished via rapid context-switching on a single processor or across multiple cores. Parallelism refers strictly to true simultaneous execution, requiring physical multi-core hardware to execute distinct tasks or subtasks at the exact same physical instant. Therefore, concurrency is about managing multiple overlapping tasks, whereas parallelism is about performing multiple computations simultaneously. **References**: GeeksforGeeks. *Difference between Concurrency and Parallelism*. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/	**Acelajado:** Concurrency is the management of multiple tasks, where multiple tasks are running at the same time and may be sharing a single core, but are switching rapidly between them, so that it appears to be running concurrently. Parallelism is about actually executing multiple tasks at the same instant, which requires multiple execution units such as CPU cores (ExplainThis, 2026). Concurrency is thus primarily a program design issue, appropriate to I/O-bound work, parallelism is primarily an execution issue, appropriate to CPU-bound work. Concurrent does not imply parallel; in a concurrent design program, additional cores can be used in parallel without it being concurrent. Threads time sliced on one core are also concurrent, but not parallel. **Reference:** ExplainThis. (2026, February 21). *What's the difference between concurrency and parallelism?* https://www.explainthis.io/en/swe/concurrency-vs-parallelism

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
Display the value of global variables at different combinations of using the join() per thread. Such that:
o Display
o T1.join()
o Display
o T2.join()
o Display
o T3.join()
Provide an analysis based on the outputs of the multi-threading exercise.

Screenshot	**Code:**
	`#include <iostream>`
	`#include <thread>`
	
	`int globalValue = 0;`
	`void add(int value) {`
	`  globalValue += value;`
	`}`
	
	`int main() {`
	`  std::thread T1(add, 15);`
	`  std::thread T2(add, 25);`
	`  std::thread T3(add, 60);`
	`  std::cout << "Display: " << globalValue << std::endl;`
	`  T1.join();`
	`  std::cout << "Display after T1.join(): " << globalValue << std::endl;`
	`  T2.join();`
	`  std::cout << "Display after T2.join(): " << globalValue << std::endl;`
	`  T3.join();`
	`  std::cout << "Display after T3.join(): " << globalValue << std::endl;`
	`  return 0;`
	`}`
	
	Output:
	`Display: 0`
	`Display after T1.join(): 100`
	`Display after T2.join(): 100`
	`Display after T3.join(): 100`
	
	`[Done] exited with code=0 in 1.613 seconds`

Analysis	**Acelajado:** For our output, the first Display can display 0 or a partial sum, since the three threads begin to run immediately they are created, and at the same time as main. Few may have added their

	values already by the time the line is printed so the number may vary from run to run. Each join() is followed by the same, or greater, value being printed, because the program waits for that thread to complete before it prints. However, the value is always 100 after T3.join(), as all three threads are finished. This indicates that the computer determines when each thread terminates and join() only determines when the main program waits. **Pangilinan:** When I looked at the output, the first Display printed 0 and every Display after a join() printed 100. The three threads (adding 15, 25 and 60) start running as soon as they are created, but main keeps going at the same time, so the first Display can run before any thread has added its value. That is why it showed 0 in our run, and the program only waits for T1, but T2 and T3 are small and fast, so they had already finished and the value was already 100.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Screenshot	Code:

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <cstdlib>

using namespace std;

void merge(vector<int>& arr, int left, int mid, int right) {
    vector<int> L(arr.begin() + left, arr.begin() + mid + 1);
    vector<int> R(arr.begin() + mid + 1, arr.begin() + right + 1);

    int i = 0, j = 0, k = left;

    while (i < L.size() && j < R.size()) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < L.size()) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < R.size()) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(vector<int>& arr, int left, int right, int threads) {
    if (left >= right) {
        return;
    }

    int mid = (left + right) / 2;

    if (threads > 1) {
        thread t(mergeSort, ref(arr), left, mid, threads / 2);
        mergeSort(arr, mid + 1, right, threads / 2);
        t.join();
    } else {
        mergeSort(arr, left, mid, 1);
        mergeSort(arr, mid + 1, right, 1);
    }

    merge(arr, left, mid, right);
}

```

```cpp
double testSort(vector<int> arr, int threads) {
    auto start = chrono::high_resolution_clock::now();
    mergeSort(arr, 0, arr.size() - 1, threads);
    auto end = chrono::high_resolution_clock::now();

    chrono::duration<double, milli> time = end - start;
    return time.count();
}

int main() {
    int n = 100000;
    vector<int> arr(n);

    srand(42);
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000000;
    }

    cout << "Array size: " << n << endl;

    double singleTime = testSort(arr, 1);
    cout << "1 thread: " << singleTime << " ms" << endl;

    int threadCounts[] = {2, 4, 8};

    for (int i = 0; i < 3; i++) {
        double multiTime = testSort(arr, threadCounts[i]);
        cout << threadCounts[i] << " threads: " << multiTime << " ms" << endl;
    }

    return 0;
}
```

Output:

```
Array size: 100000
1 thread: 52.5233 ms
2 threads: 28.3899 ms
4 threads: 17.0386 ms
8 threads: 13.1886 ms
```

Analysis

Acelajado: I implement multi-threading of merge sort in this activity, which is sorting left side of the array in a new thread and sorting right side of the array in the main thread and finally merging two halves. I have tested on 100,000 numbers with 1, 2, 4 and 8 threads and have calculated the running times. The results indicated that if more threads are used, the sorting process can be faster but this gain is not always significant and will not increase in proportion to the number of threads added, since the merge step will be performed using only one thread, and the creation of many threads will lead to additional overhead. The time complexity remains O(n log n), because the total work is not decreased, only the running time. The optimal number of threads depends on the number of CPU cores.

Pangilinan:
For this part I used merge sort and sorted 100,000 random numbers using 1, 2, 4 and 8 threads. In the code, each time mergeSort splits the array, it sorts the left half in a new thread and the right half in the current thread, then joins and merges the two halves. The threads count is divided by 2 on each split, so the work is spread across the threads. The output shows the time going down as threads were added: 52.5233 ms for 1 thread, 28.3899 ms for 2 threads, 17.0386 ms for 4 threads and 13.1886 ms for 8 threads.

Conclusion

Acelajado: In conclusion, I discovered that multithreading allows a program to execute multiple threads simultaneously, thereby improving its speed and responsiveness. In our exercise, the three threads multiplied their values together pretty much simultaneously, and the total sum was always 100. Also, an important thing we learned is that join() forces wait for the thread to complete, which means that the final value is only used when the work is finished. In general, this exercise has helped us to understand the working of threads in C++ and the significance of join() in multithreading.

Pangilinan:
Multithreading allows a program to execute multiple tasks simultaneously, significantly enhancing both speed and responsiveness. Through this activity, I observed that threads begin running as soon as they are created and execute independently, requiring control functions like `join()` to force the main program to wait for their completion before safely continuing. Sharing data across these concurrent threads introduces severe timing risks without synchronization, making data safety a critical priority in parallel programming. Furthermore, while splitting large computational workloads across multiple threads can dramatically reduce execution time, performance gains eventually hit a limit due to thread management overhead and inherently sequential steps that cannot be run in parallel.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Use for grammar only

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.